Stage 6: SOAP Note Generation

Part 1 - approaches, design and the structured fact builder (6a)
Project: Doctor–Patient Conversation Audio -> SOAP Notes

Goal: turn the role-labelled conversation and BioBERT entities into a SOAP note that is **grounded** (every statement traceable to the transcript), **safe** (no invented diagnoses or treatments) and **readable**. Built in three steps: **6a** structured facts (rules, done), **6b** note writing with a pretrained Transformer language model, **6c** grounding check.

Contents

1. What a SOAP note is
2. Problems any approach must solve
3. Four possible approaches
4. Our design: hybrid, grounded generation
5. Why this design (and the resume)
6. 6a - the structured fact builder
7. 6a results on our transcripts
8. 6b / 6c - planned language-model step and grounding check
9. Evaluation of SOAP notes
10. Limitations
11. Interview questions and answers

1. What a SOAP note is

Section	Contains	Comes mostly from
S - Subjective	symptoms, duration, severity, history, current medications, concerns, denials	PATIENT statements and answers
O - Objective	examination findings, vital signs, measurements	DOCTOR (examining / measuring)
A - Assessment	diagnosis or clinical impression	DOCTOR
P - Plan	treatment, prescriptions, tests, referrals, follow-up	DOCTOR

The SOAP format (Weed, 1960s) is the standard structure for clinical progress notes. Automating it is the core of 'ambient clinical documentation' / AI-scribe products.

2. Problems any approach must solve

Problem	Example	Why it matters
Symptom ownership via Q/A	DOCTOR: 'Any nausea?' PATIENT: 'Yes.'	the entity is in the doctor's sentence, but the PATIENT has it
Negation	'No, I haven't had any fever.'	must be recorded as denied, not present
Uncertain answers	'Only when I climb stairs.'	neither yes nor no - keep the quote
Multi-sentence turns	'It looks like a migraine. I will prescribe ...'	one turn holds A and P
No medical decisions	-	A and P may only contain what the doctor said
Traceability	-	a clinician must be able to verify each line

3. Four possible approaches

	1. Rules + template	2. LLM prompt	3. Fine-tuned seq2seq	4. Hybrid (ours)
How	map entities/cues to sections, fill a template	transcript -> pretrained LLM -> note	train Flan-T5 / BioBART on dialogue->note pairs	rules build facts; pretrained LM writes from facts only; grounding check
Data needed	none	none	ACI-Bench / MTS-Dialog	none
Readability	mechanical	natural	medium	natural
Hallucination risk	none	**high**	medium	low (checked)

	1. Rules + template	2. LLM prompt	3. Fine-tuned seq2seq	4. Hybrid (ours)
Traceability	full	weak	weak	full
Privacy	local	local or cloud	local	local

- **1. Rules + template**: transparent and safe, but reads like a form and misses anything the rules don't cover.
- **2. LLM prompt**: how commercial AI scribes work; fluent and context-aware, but can add facts nobody said (hallucination), and cloud APIs send patient data to a third party.
- **3. Fine-tuned sequence-to-sequence model**: an encoder-decoder (T5, BART) trained on dialogue/note pairs (ACI-Bench ~200 encounters, MTS-Dialog ~1,700 dialogue snippets). Measurable and local, but small datasets give incomplete notes and it can still hallucinate.
- **4. Hybrid**: combine the safety of 1 with the fluency of 2.

4. Our design: hybrid, grounded generation

```
role-labelled transcript + BioBERT entities (Stage 5)
        |
[6a] build_soap_facts.py  (rules, deterministic)
     Q/A pairing -> negation -> attribute linking -> S/O/A/P assignment
     -> soap_facts.json (every fact + source sentence) + template note (baseline)
        |
[6b] pretrained Transformer LM, run locally
     prompt: 'Write a SOAP note using ONLY these facts'
        |
[6c] grounding check: every clinical term in the note must exist in the facts
     -> unsupported content flagged as possible hallucination
```

This is called **grounded generation** (related to retrieval-augmented generation): the language model is not asked what it knows, only to rephrase evidence it is given.

5. Why this design (and the resume)

- Resume bullet: *'Generated structured SOAP notes from consultations using pretrained Transformer-based language models'*. The hybrid uses a pretrained Transformer LM for writing, so the claim is accurate; rules alone would not be.
- Fine-tuning a note generator is not needed for 'pretrained' and would be a much bigger project; it is a good 'future work' answer.
- Local model: no patient data leaves the machine.
- The template note from 6a is a **baseline** - we can show what the language model adds.

6. 6a - the structured fact builder (build_soap_facts.py)

Step	Rule	Example
Q/A pairing	entities in a DOCTOR question + next turn's answer. 'yes/yeah/I do...' -> present, 'no/never/I haven't...' -> absent, else unclear (quoted)	'Any fever?' + 'No, ...' -> fever absent

Step	Rule	Example
Negation (NegEx-style)	cue (no, not, never, denies, haven't...) within 40 characters before the entity, no comma/'but' in between	'I haven't had any fever' -> absent
Patient questions	entities inside a patient's question are not reports; the question becomes a concern	'Should I worry about the chest pain?'
Attribute linking	Duration/Severity -> nearest symptom; Dosage/Frequency -> nearest medication (same sentence)	metformin + '500 mg twice daily'
Doctor sentences	split turns into sentences (not at decimals); each sentence -> P if plan cue, else O if exam cue, else A if assessment cue or Disease entity	'It looks like a migraine.' -> A
Generic words	'medication', 'symptoms' ... are ignored as facts	'Are you taking any medication?'
Merging	same section+type+text+status -> one fact, all evidence kept	fever from Q/A and answer

Each fact stores: section, type, text, status (present / absent / unclear / asked / stated by doctor), attributes, the BioBERT label, the rule used, and **evidence** (speaker, sentence, utterance index, timestamps). Utterances that produced no fact are listed too, so nothing is silently dropped.

> **Safety rule:** Assessment and Plan are filled only from DOCTOR sentences, quoted. The system never diagnoses or prescribes; missing values (e.g. a blood-pressure number nobody said) are never written.

7. 6a results on our transcripts

```
SUBJECTIVE
  Reported symptoms/conditions: cough (duration: two weeks); pain
  Denied: fever
  Asked, answer unclear: shortness of breath
  Current medications: metformin (dosage: 500 mg twice daily)
  Patient questions/concerns: "Should I worry about the chest pain?"
OBJECTIVE
  - "Let me listen to your chest."
  - "Your temperature is 37.2 degrees and your lungs sound clear."
ASSESSMENT (as stated by the doctor)
  - bronchitis  ["This looks like acute bronchitis."]
PLAN (as stated by the doctor)
  - "I will prescribe an inhaler."
  - "Come back in one week if it is not better."
```

- Sample 2: nausea and vomiting correctly given to the patient through 'Yes.'; migraine -> A; prescription and return advice -> P (after splitting the doctor's turn into sentences).
- Bugs found while testing and fixed: whole turns landing in both A and P; '37.2' split into two sentences; duplicate facts; a patient's question creating a 'present' symptom.
- Visible upstream gaps: NER missed 'paracetamol' in sample 1, so its dose appears as an unattached detail; 'diabetes' and 'chest' in 'chest pain' were missed. The facts layer shows these gaps instead of hiding them.

8. 6b / 6c - planned language-model step and grounding check

Candidate (checked Nov 2026)	Size	License	Notes
Qwen3-4B-Instruct-2507	4.0 B (~8 GB)	Apache-2.0, not gated	recommended: permissive, recent
Phi-4-mini-instruct	3.8 B (~7.7 GB)	MIT, not gated	strong alternative
Qwen2.5-3B-Instruct	3.1 B	Qwen research license	restrictive license
Llama-3.2-3B / Gemma-3-4B	3.2 / 4.3 B	custom, gated	needs access approval
Flan-T5-large	0.8 B	Apache-2.0	older encoder-decoder; weaker notes

- **Decoder-only LLM** (Qwen, Phi, Llama): generates text left-to-right, predicting the next token; instruction-tuned versions follow prompts. **Encoder-decoder** (T5, BART): encodes the input, then decodes the output - the classic summarisation architecture.
- Prompt = system rules (use only given facts, quote doctor's assessment and plan, write 'not documented' if missing) + the facts JSON. Greedy decoding (no sampling) for reproducible output.
- 6c grounding check: extract clinical terms from the generated note (with our BioBERT model) and verify each appears in the facts; report coverage (facts used) and unsupported terms (possible hallucinations).

9. Evaluation of SOAP notes

- **Coverage / recall of facts**: share of extracted facts that appear in the note.
- **Hallucination rate**: statements or entities with no support in the transcript.
- **ROUGE** (n-gram overlap) or BERTScore against clinician-written notes, e.g. on ACI-Bench.
- **Section accuracy**: is each fact in the right S/O/A/P section?
- **Clinician review**: the gold standard - correctness, completeness, usefulness.

10. Limitations

- Rules cover common phrasings only (English, simple yes/no answers, short negation window).
- Q/A pairing assumes the answer is the next turn; real conversations interleave topics.
- Upstream errors (ASR, diarization, roles, NER) flow into the facts; NER misses mean missing facts.
- Section cues are keyword-based: 'take rest' vs 'take a history' can confuse plan detection.
- O section often contains only 'examination performed' because values are not spoken aloud.

11. Interview questions and answers

Q1. How do you generate SOAP notes in your project?

A hybrid: rules turn the role-labelled transcript and BioBERT entities into structured S/O/A/P facts with source sentences; a locally-run pretrained Transformer LM writes the note from only those facts; a grounding check flags any clinical term not supported by the facts.

Q2. Why not just give the transcript to an LLM?

LLMs can hallucinate symptoms, diagnoses or doses. Constraining generation to extracted facts and verifying the output makes every statement traceable, which clinical documentation requires.

Q3. How do you know who has a symptom?

Speaker isn't enough: 'Any nausea?' is said by the doctor. We pair doctor questions with the next patient answer and set the status from the answer: yes -> present, no -> absent, otherwise unclear with the quote.

Q4. How do you handle negation?

NegEx-style: a negation cue shortly before the entity with no clause boundary in between marks it absent. Denied findings are listed separately - they matter clinically ('denies chest pain').

Q5. How do you make sure the system doesn't make medical decisions?

Assessment and Plan are filled only from the doctor's own sentences, quoted. Nothing is inferred; missing values are never filled in.

Q6. What is grounded generation?

Generating text conditioned on supplied evidence and restricted to it, then verifying the output against that evidence - closely related to retrieval-augmented generation.

Q7. Why a local model instead of a cloud API?

Patient conversations are protected health information; running locally avoids sending them to third parties. A cloud API would need a healthcare data agreement (e.g. HIPAA BAA).

Q8. Which language model and why?

A small instruction-tuned decoder model such as Qwen3-4B-Instruct (Apache-2.0, ~4B parameters): good instruction following, permissive license, fits on a laptop. Larger models write better but need more memory.

Q9. Encoder-decoder vs decoder-only - which would you use?

Encoder-decoders (T5/BART) are natural for summarisation when fine-tuning; instruction-tuned decoder-only LLMs are better zero-shot. Since we use a pretrained model without fine-tuning, decoder-only is stronger.

Q10. How would you evaluate the notes?

Fact coverage, hallucination rate, section accuracy, ROUGE/BERTScore against reference notes (ACI-Bench), and clinician review.

Q11. What did the template note teach you?

It's a safe baseline that shows exactly what was extracted; comparing it with the LM note isolates what the language model adds (fluency) and risks (unsupported content).

Q12. What bugs did you find while building the fact layer?

Multi-sentence doctor turns landing in both A and P, decimals split as sentence ends, duplicate facts from Q/A plus answer, and entities in a patient's question being recorded as present symptoms.

Q13. How would you improve it?

Learned section and assertion classifiers instead of rules, better Q/A linking across turns, entity linking to SNOMED/RxNorm, and fine-tuning a generator on ACI-Bench for comparison.